



ASCII

SEARCHING



PENGERTIAN SEARCHING

Algoritma search adalah serangkaian langkah atau instruksi yang digunakan untuk mencari elemen atau informasi tertentu di dalam suatu dataset. Tujuannya adalah untuk menemukan posisi atau keberadaan elemen yang dicari. Dalam pemrograman, algoritma search menjadi salah satu teknik penting dalam menyelesaikan berbagai masalah.

1. Binary search

Binary Search (pencarian biner) adalah salah satu algoritma pencarian terhadap data yang sudah terurut. Mengapa disebut binary search? Karena algoritma ini membagi array menjadi dua bagian di setiap langkah pencariannya. Algoritma ini menggunakan prinsip divide and conquer, yaitu membagi masalah besar menjadi masalah yang lebih kecil hingga ditemukan solusi. Saat melakukan pencarian, algoritma ini memeriksa elemen tengah dari daftar, lalu menentukan apakah target lebih kecil, lebih besar, atau sama dengan elemen tersebut.

Cara kerja Binary Search:

1. Hitung indeks tengah (mid) dari array dengan rumus:
$$MID = (low + high) / 2$$
2. Bandingkan nilai pada indeks mid dengan nilai yang dicari (kunci)
3. Jika nilai di mid == kunci, maka pencarian selesai
4. Jika nilai di mid > kunci, cari di bagian kiri (low tetap, high = mid - 1)
5. Jika nilai di mid < kunci, cari di bagian kanan (low = mid + 1, high tetap)
6. Ulangi seluruh proses di atas hingga elemen ditemukan atau low > high yang artinya elemen tidak ada dalam array.

Berikut ilustrasi Binary Search:



Adapun untuk implementasinya pada kode di bawah ini:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Fungsi binary search yang mengembalikan indeks atau -1 jika tidak
ditemukan
int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low ≤ high) {
        int mid = low + (high - low) / 2; // Menghindari potensi
integer overflow
    }
```



```
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    // Data harus dalam keadaan terurut (sorted)
    vector<int> kumpulanData = {10, 23, 35, 48, 50, 72, 85, 99};
    int target;

    cout << "Cari angka dalam array: ";
    cin >> target;

    int hasil = binarySearch(kumpulanData, target);

    if (hasil != -1) {
        cout << "Angka " << target << " ditemukan pada indeks: " <<
hasil << endl;
    } else {
        cout << "Angka " << target << " tidak ada dalam data." <<
endl;
    }

    return 0;
}
```

Pada binary search kompleksitas algoritma dapat kita bagi menjadi 2, yaitu best case dan worst case, maksud best case di sini adalah ketika kasus terbaik dimana target yang dicari



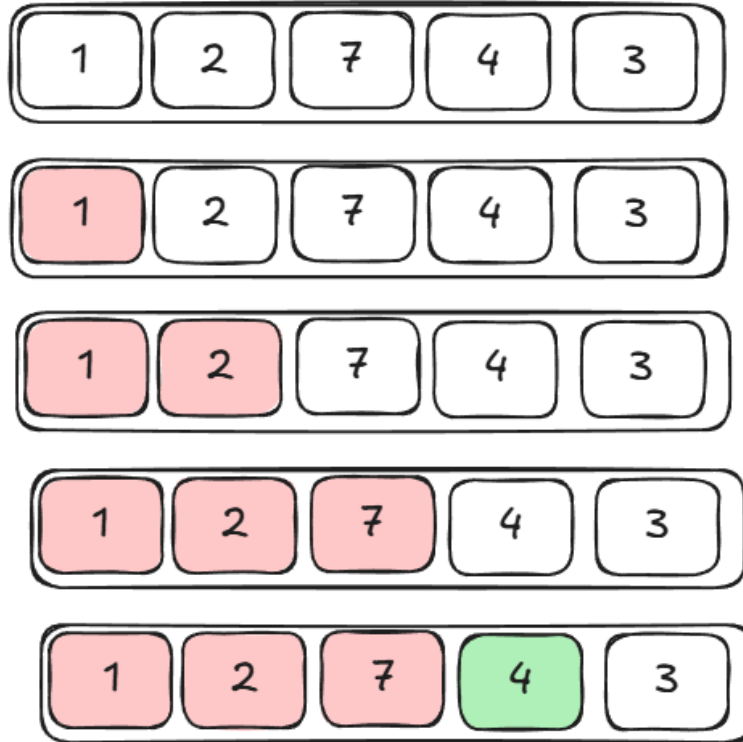
tepat berada di tengah yang hanya membutuhkan 1 operasi saja hal ini akan menimbulkan nilai O konstan yaitu $O(1)$, sedangkan worst casenya ketika elemen yang dicari berada pada indeks yang paling ujung sehingga kompleksitasnya adalah $O(\log n)$.

2. Linear Search

Linear search merupakan salah satu algoritma yang paling mudah untuk dipahami, karena konsep daripada algoritma ini serupa dengan perulangan dimana mengandalkan jumlah data yang ada untuk ditelusuri satu persatu, algoritma ini akan efisien dalam data kecil, tetapi akan sangat lambat jika digunakan dalam data yang besar. Dalam pemahaman ini secara tidak langsung bahwa kita dapat memahami bahwa pertumbuhan linear search adalah linier atau bergantung pada jumlah elemen (n).



linear search
target = 4



```
#include <iostream>
using namespace std;

int main() {
    int data[] = {12, 45, 7, 23, 56, 10, 34};
    int n = sizeof(data) / sizeof(data[0]); // Menghitung jumlah
elemen
    int cari;
    bool found = false;

    cout << "Masukkan angka yang ingin dicari: ";
    cin >> cari;
```



```
// Proses Linear Search
for (int i = 0; i < n; i++) {
    if (data[i] == cari) {
        cout << "Data ditemukan pada indeks ke-" << i << endl;
        found = true;
        break; // Berhenti jika sudah ketemu
    }
}

if (!found) {
    cout << "Data tidak ditemukan dalam array." << endl;
}

return 0;
}
```



3. Jump Search

Jump Search adalah algoritma pencarian yang bekerja dengan cara melompati beberapa elemen untuk menemukan nilai target dalam array yang terurut.

Cara kerja:

- Melompat beberapa langkah terlebih dahulu untuk mempercepat pencarian.
- Setelah mendekati posisi target, gunakan Linear Search dalam blok tersebut.

Istilah yang ada di jump search:

- Prev
 - Berupa index yang nyimpen posisi awal dari blok yang diperiksa
 - Setelah menemukan blok yang mungkin berisi elemen yang dicari, pencarian dilakukan secara linear mulai dari prev.
 - Nilai prev akan menyimpan nilai step sebelumnya di setiap iterasi.
- Step
 - step dihitung dari akar panjang array
 - Sebagai akhir dari blok yang diperiksa
 - Nilai step akan bertambah sebanyak nilai akar dari panjang array selama nilai dari blok kurang dari elemen yang dicari.
- Blok
 - Blok adalah bagian dari array yang diperiksa dalam satu lompatan sebelum melakukan pencarian linear.
 - Array dibagi menjadi blok berukuran $\text{step} = \sqrt{n}$.
 - Blok yang berisi elemen yang dicari dan melakukan Linear Search dalam Blok.



JUMP SEARCH

TARGET 19

jumlah array(n)=16

jump= $\sqrt{n}=4$

pisahkan setiap blok sesuai dengan jump



bandingkan prev dan high blok 1

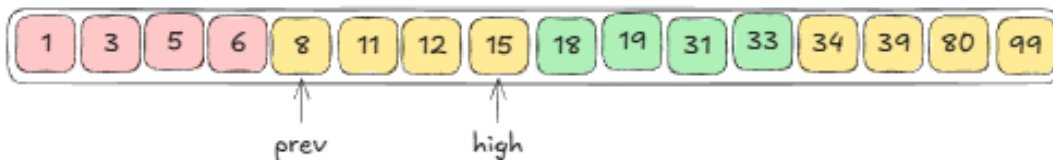
1 dan 6 lebih kecil dari 19, tidak mungkin terdapat pada blok ini



sekarang pindah prev dan high

bandingkan prev dan high blok 2

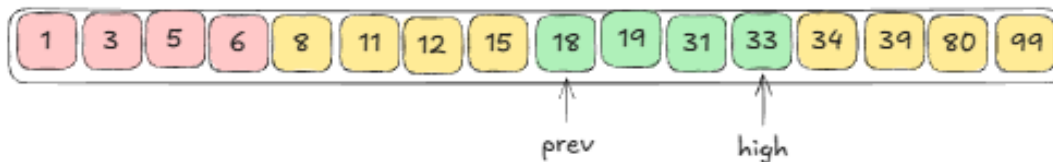
8 dan 15 lebih kecil dari 19, tidak mungkin terdapat pada blok ini



sekarang pindah prev dan high

bandingkan prev dan high blok 3

$18 < 19 < 33$ berarti kemungkinan ada pada blok ini, pisahkan lalu gunakan linear search



elemen ditemukan



```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;

int jumpSearch(int arr[], int x, int n) {
    // Menentukan ukuran blok lompatan
    int step = sqrt(n);
    int prev = 0;

    // Melompat ke depan selama elemen di indeks lompatan kurang dari
    target
    while (arr[min(step, n) - 1] < x) {
        prev = step;
        step += sqrt(n);
        if (prev ≥ n) return -1; // Jika sudah di ujung tapi belum
ketemu
    }

    // Melakukan Linear Search di dalam blok yang sudah ditemukan
    while (arr[prev] < x) {
        prev++;
        // Jika sampai ke blok berikutnya atau ujung array, berarti
tidak ada
        if (prev == min(step, n)) return -1;
    }

    // Jika elemen ditemukan
    if (arr[prev] == x) return prev;

    return -1;
}
```



```
int main() {  
    int data[] = {2, 5, 8, 12, 19, 22, 26, 29, 35, 40};  
    int n = sizeof(data) / sizeof(data[0]);  
    int cari = 26;  
  
    int hasil = jumpSearch(data, cari, n);  
  
    if (hasil  $\neq$  -1) {  
        cout << "Data ditemukan pada indeks ke-" << hasil << endl;  
    } else {  
        cout << "Data tidak ditemukan" << endl;  
    }  
  
    return 0;  
}
```

Algoritma ini memiliki kompleksitas $O(\sqrt{n})$. Jika ukuran lompatan adalah m , maka kita melakukan n/m lompatan. Kasus terburuknya adalah kita melakukan n/m lompatan dan $m-1$ perbandingan linear. Dengan memilih $m = \sqrt{n}$, kita mendapatkan total langkah minimal.



4. Interpolation Search

Interpolation Search adalah algoritma yang bekerja dengan cara menebak posisi elemen yang dicari berdasarkan nilai elemen.

Interpolation Search digunakan untuk mencari elemen dalam array yang terurut.

Cara kerja:

- Pencarian menggunakan rumus interpolasi.
- Jika elemen saat ini == elemen yang dicari, elemen ditemukan.
- Jika elemen saat ini < elemen yang dicari, lanjutkan pencarian di sub-array kanan.
- Jika elemen saat ini > elemen yang dicari, lanjutkan pencarian di sub-array kiri.

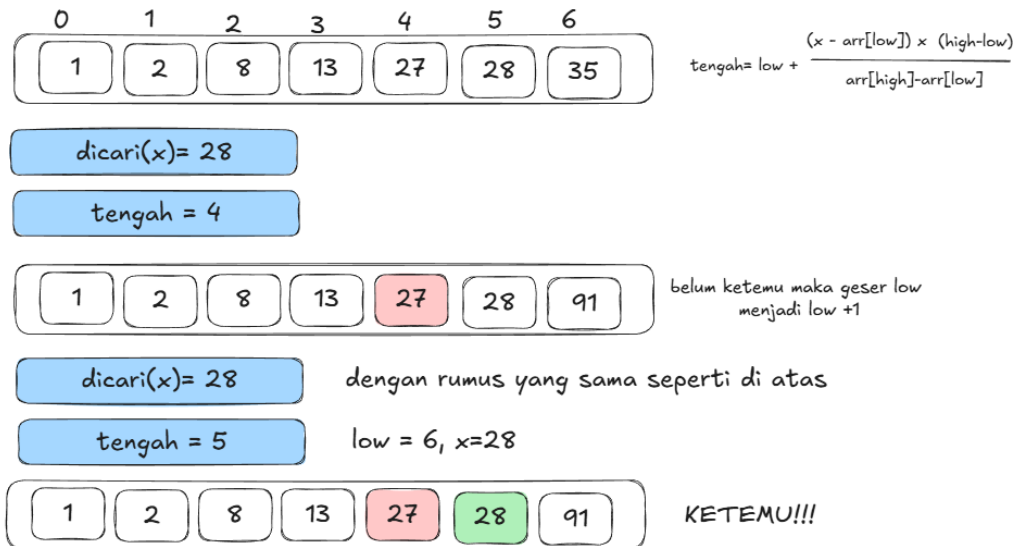
Rumus Interpolasi:

$$pos = low + \frac{(x - arr[low]) * (high - low)}{arr[high] - arr[low]}$$

- pos: nilai tengah dari array.
- low: index terkecil dalam array.
- high: index terbesar dalam array.



Interpolation Search



```
#include <iostream>
#include <vector>

using namespace std;

int interpolationSearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;

    // Syarat: x harus berada di dalam rentang nilai array
    while (low ≤ high && x ≥ arr[low] && x ≤ arr[high]) {

        // Kasus khusus jika elemen low dan high sama untuk
        // menghindari pembagian dengan nol
        if (low == high) {
            if (arr[low] == x) return low;
            return -1;
        }
    }
}
```



```
// Rumus estimasi posisi (Interpolasi)
int pos = low + (((double)(high - low) / (arr[high] -
arr[low])) * (x - arr[low]));

// Jika target ditemukan
if (arr[pos] == x) {
    return pos;
}

// Jika target lebih besar, cari di sisi kanan
if (arr[pos] < x) {
    low = pos + 1;
}

// Jika target lebih kecil, cari di sisi kiri
else {
    high = pos - 1;
}
}
return -1;
}

int main() {
    // Data dari gambar: [1, 2, 8, 13, 27, 28, 35]
    int data[] = {1, 2, 8, 13, 27, 28, 35};
    int n = sizeof(data) / sizeof(data[0]);
    int cari = 28;

    int hasil = interpolationSearch(data, n, cari);

    if (hasil != -1) {
        cout << "Data " << cari << " ditemukan pada indeks ke-" <<
hasil << endl;
    } else {
        cout << "Data tidak ditemukan." << endl;
    }
}
```



```
return 0;  
}
```

Jika elemen-elemen dalam array terdistribusi secara merata (linear atau uniform), algoritma ini jauh lebih cepat daripada Binary Search. Sebagai contoh, untuk 1.000.000 data, Binary Search butuh sekitar 20 perbandingan, sementara Interpolation Search hanya butuh sekitar 4 hingga 5 perbandingan sehingga kompleksitasnya adalah $O(\log(\log(n)))$.



STUDI KASUS

1. Mas Rusdi baru saja menerima kiriman berbagai jenis anggrek. Untuk menarik pelanggan di Pontianak, ia ingin menampilkan daftar harga dari yang termurah hingga termahal. Pelanggan sering datang menanyakan apakah ada anggrek dengan harga tertentu yang sesuai dengan *budget* mereka.

Data Inventaris (Harga dalam ribuan Rp):

- Anggrek Hitam: 250
- Anggrek Bulan: 85
- Anggrek Dendrobium: 45
- Anggrek Vanda: 120
- Anggrek Tanah: 30

2. Mas Rehan dipercaya untuk mengelola stok harian di gerai Puding Bali. Karena permintaan sedang tinggi, Mas Rehan harus memastikan stok puding dicatat dengan rapi. Mas Rehan ingin mengurutkan stok dari yang paling sedikit agar ia tahu varian mana yang harus segera diproduksi ulang oleh tim Kakangku.

Data Stok Puding (dalam cup):

- Puding Cokelat: **45**
- Puding Mangga: **60**
- Puding Pandan: **15**
- Puding Susu: **20**